



《软件测试》

第十二章 公有云测试 质量评估



授课人：高利鹏

日期：2025年12月24日

目 录

CONTENTS

01

云测试概念

02

云可靠性度量

03

云平台的安全测试及安全
度量

04

小结

学习目标

- ◆ 理解云计算和云测试的概念
- ◆ 掌握云可靠性度量的方法
- ◆ 掌握云平台安全性测试的方法

目 录

CONTENTS

01

云测试概念

02

云可靠性度量

03

云平台的安全测试及安全
度量

04

小结

1.1 云计算

从云部署的角度定义



从云计算服务的角度定义

基础设施即服务 (Infrastructure as a Service, IaaS)

平台即服务 (Platform as a Service, PaaS)

软件即服务 (Software as a Service, SaaS)

1.2 云测试

云测试的两层含义

- 有效利用云计算环境资源对其他软件进行测试，即基于云计算的测试
- 针对部署在“云”中的软件进行测试，即面向云计算的测试。

几种典型的云测试

➤ 云环境中的测试

测试涉及云环境资源的调度、优化、建模等方面问题。

➤ 针对“云”的测试

主要是面向云的测试，主要对云计算内部结构、内部框架、资源配置等云环境内部的组成要素进行测试，并主要对以下四个方面测试。

- (1) 功能性测试
- (2) 性能测试
- (3) 安全性测试
- (4) 兼容性和互操作性测试

目 录

CONTENTS

01

云测试概念

02

云可靠性度量

03

云平台的安全测试及安全
度量

04

小结

2.1 软件可靠性

软件可靠性定义

软件产品在规定的条件下和规定的时间区间完成规定功能而不发生软件失效的概率。

1. 软件可靠性基础理论

度量软件可靠性的本质是计算软件系统发生故障的概率。考虑系统在 t 时刻发生故障的概率 P ，该概率可表达为

$$P(t \leq T \leq t + \Delta t) \equiv \text{软件失效概率} \leq T \leq t + \Delta t$$

其中， T 表示软件失效出现的时间。这个概率可以表示为

$$P(t \leq T \leq t + \Delta t) = f(t)\Delta t = F(t + \Delta t) - F(t)$$

2.1 软件可靠性

通过 $F(t)$ 和 $f(t)$ 可进一步定义可靠函数 $R(t)$,表示在 t 时刻前, 系统不发生软件失效的概率, 定义公式为

$$R(t) = P(T > t) = 1 - F(t) = \int_t^{\infty} f(x)dx$$

在此基础上, 定义失效概率 $failure\ rate$,其含义为系统的首次软件失效发生在 t 时刻到 $t + \Delta t$ 时间段内, 其概率表达式为:

$$\begin{aligned} failure\ rate &\equiv \frac{P(t \leq T < t + \Delta t | T > t)}{\Delta t} = \frac{P(t \leq T < t + \Delta t)}{\Delta t P(T > t)} \\ &= \frac{F(t + \Delta t) - F(t)}{\Delta t R(t)} \end{aligned}$$

当 Δt 趋于0时, 可以定义风险函数 $z(t)$,其含义为: 系统在 t 时刻首次出现系统失效概率, 数学表达式为

$$z(t) = \lim_{\Delta t \rightarrow 0} \frac{F(t + \Delta t) - F(t)}{\Delta t R(t)} = \frac{f(t)}{R(t)}$$

其结果如下:

$$f(t) = z(x) \exp \left[- \int_0^t z(x) dx \right] \quad \text{或} \quad R(t) = e^{- \int_0^t z(x) dx}$$

2.1 软件可靠性

在此基础上，定义平均故障间隔时间（Mean Time Between Failures, MTBF）作为可靠性的度量，其公式如下：

$$MTBF = \int_0^{\infty} R(x)dx$$

下面以Goel-Okumoto软件可靠度预测模型为例，阐述估计风险函数的方法。

- 假设系统在各个时间段内观测到的系统失效相互独立，将时刻 t 时所观测到的系统失效总数表示为 $N(t)$,
- 假设系统在任何时刻发生软件失效的概率与系统当前蕴含的总故障数成正比，设系统总故障数为 a ，系统的故障检测速率为 b ，有如下公式：

$$m(t + \Delta t) - m(t) = b\{a - m(t)\}\Delta t + o(\Delta t)$$

$$\text{当}\Delta t\text{趋于}0\text{时} \quad m'(t) = ab - bm(t)$$

2.1 软件可靠性

根据初始条件

$$m(t) = \begin{cases} 0, t = 0 \\ a, t \rightarrow \infty \end{cases}$$

解得微分方程

$$m(t) = a(1 - e^{-bt})$$

定义 $\lambda(t)$ 表示系统瞬时的软件失效发生率 $\lambda(t) \equiv m'(t) = abe^{-bt}$

回顾 $R(t)$ 的定义:

$$R(t) = e^{-\int_0^t z(x) dx}$$

考虑软件可靠性增长的过程是系统状态在时间 s 上不断变化的过程, 在时间 s 时, 系统状态为 $C(s)$ 。 $Z(t | s)$ 表示系统状态为 $C(s)$ 时, 在0到 t 时间内, 系统发生软件失效的概率值为 $\lambda(t)$

2.1 软件可靠性

2. 传统软件可靠性模型

常用的3大类软件可靠性模型

- 输入域的软件可靠性模型
- 时间域的软件可靠性模型
- 结合时间域和输入域的可靠性模型

2.1 软件可靠性

2. 传统软件可靠性模型

(1) 输入域的软件可靠性模型

输入域的软件可靠性模型 (Input Domain Reliability Model) 的代表是纳尔逊 (Nelson) 模型。该模型假设随机地在程序的输入域上取 n 个不同输入并执行, 其中 f 个输入引起程序失效, 则估计的软件可靠性为

$$\hat{R} = 1 - \frac{f}{n} = \frac{n - f}{n}$$

这类软件可靠性模型还有其他拓展, 如布朗-理珀 (Brown-Lipow) 模型, 该模型将软件的输入域划分为若干个子域, 估计的软件可靠性可以表示为

$$\hat{R} = 1 - \sum_{j=1}^N \left(\frac{f_j}{n_j} \right) P(E_j)$$

2.1 软件可靠性

2. 传统软件可靠性模型

(2) 软件可靠性增长模型

软件可靠性增长模型（Software Reliability Growth Model, SRGM）通常使用一个非齐次泊松过程（Non Homogeneous Poisson Process, NHPP）来表示：

$$P[X(t) = n] = \frac{[m(t)]^n e^{-m(t)}}{n!}$$

其中， $X(t)$ 表示在 t 时刻累计检测到的系统故障数， $m(t)$ 为均值函数。

NHPP类SRGM的代表有歌尔-大本（Goel-Okumoto, GO）模型、S型模型、穆萨-大本（Musa-Okumoto, MO）模型等

GO模型将累计发现的系统故障数和系统使用时间的关系表示为指数型的关系，其均值函数为

$$\text{GO: } m(t) = N(1 - e^{-bt})$$

2.1 软件可靠性

2. 传统软件可靠性模型

(2) 软件可靠性增长模型

S型模型将累计故障数在时间 t 上的变化过程表示为S型曲线，其均值函数为

$$m(t) = N(1 - (1 + bt)e^{-bt})$$

MO模型认为应当使用CPU时间而非自然时间作为可靠性度量的时间单位，使用符号表示累计CPU执行时间。其均值函数为：

$$\text{MO: } m(t) = \frac{1}{\theta} \log(\lambda_0 \theta \tau + 1)$$

2.1 软件可靠性

2. 传统软件可靠性模型

(3) 结合输入域和时间域的软件可靠性模型

结合输入域和时间域的软件可靠性模型利用树状结构组织管理测试数据，该模型的建立过程如下：

步骤1：从根节点出发自上向下的对当前树的叶子节点执行分裂操作，直到叶子节点上包含的用例数小于阈值或其他终止条件。

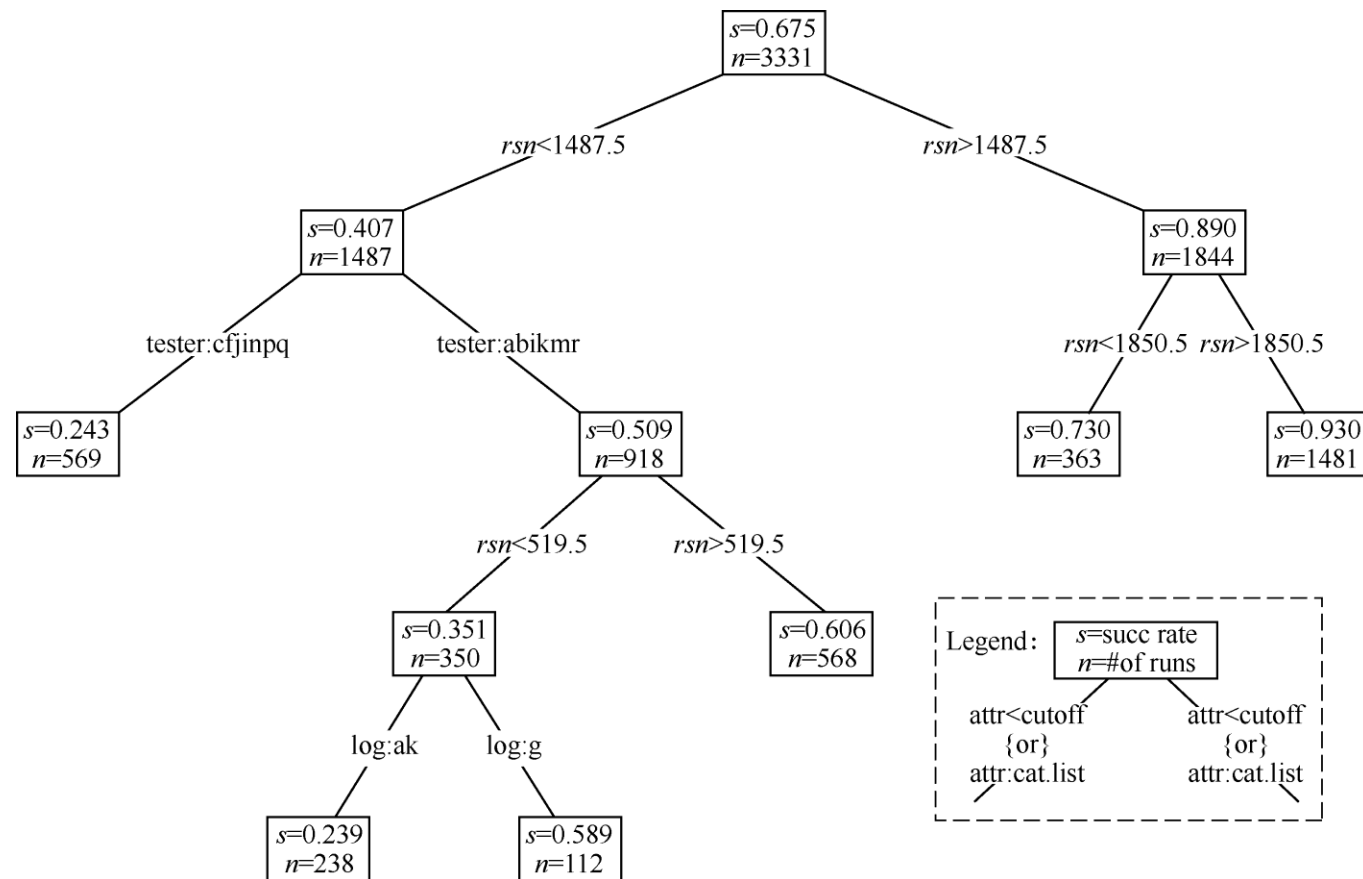
步骤2：对于某一待分裂的节点，根据某一度量属性，如执行时间、负责人、影响模块、执行结果、执行时长等，将该节点的所有数据元素二分。

步骤3：执行叶子节点的分裂操作时，根据测试用例执行结果，选取将该节点二分的最优策略，即使分裂出的两组数据集的组内方差和最小。

2.1 软件可靠性

2. 传统软件可靠性模型

建立好的树状模型



使用Nelson模型计算不同节点上的软件可靠性，可有效地识别出系统高风险区域。通过在不同检测周期建立的树状模型，可以有效地检测系统可靠性增长过程。

2.1 软件可靠性

3. Web软件可靠性模型

Web类型的软件软件失效模式、系统工作负载模式，以及潜在的软件可靠性提升都与传统的软件系统不同。其系统外部失效由于用户数量大、系统使用状态复杂等原因而难以观测。因此，可以通过度量其内部的软件缺陷来度量系统的可靠性。

定义Web服务的软件故障表

Type	Description
A	Permission defined
B	No such file or directory
C	Stale NFS file handle
D	Client denied by server configuration
E	File does not exist
F	Invalid method in request
G	Invalid URL in request connection
H	Mod_mime_magic
I	Request failed
J	Script not found or unable to start
K	Connection reset by peer

2.1 软件可靠性

3. Web软件可靠性模型

在某一网站近一个月的真实运行统计中，以上各类型软件故障的数量分布如表所示，类型E是用户所请求的文件不存在，这类故障类型的占比90%以上。

Error Type	A	B	C	D	E	F	G	H	I	J	K	Total
Number of error	2079	14	4	2	28631	0	1	1	1	27	0	30760

详细分析该类型的软件故障

File Type	Error Share (%)	Error Rate (%)
Page	61.06	3.80
Graph	33.84	0.90
Document	3.54	2.51
Other	1.56	10.16

2.1 软件可靠性

3. Web软件可靠性模型

分析各类软件故障的占比（Error Share）和发生概率（Error Rate）

File Type (attribute1)	Owner Type (attribute2)			
	Official		Personal	
	Error Share (%)	Error Rate (%)	Error Share (%)	Error Rate (%)
Page	25.55	2.03	74.45	5.15

通过分析各类软件故障的占比（Error Share）和发生概率（Error Rate），可以综合评估各类软件故障对系统可靠性的影响。

2.1 软件可靠性

4. 新型云环境下的软件可靠性模型

可靠性是指任何与计算相关的组件（软件、硬件或者网络）能根据其规格连续执行。云可靠性是指云资源能否在其标准内持续执行。

基于跳跃扩散模型的新型软件可靠性管理方法

设 $M(t)$ 为测试时间 t 软件系统剩余故障数，并假设 $M(t)$ 具有连续的实值，可以得到

$$\frac{dM(t)}{dt} = -b(t)M(t)$$

其中， $b(t)$ 是测试时间 t 每个故障单位时间的故障检测率，是一个非负函数。上式可定义为

$$\frac{dM(t)}{dt} = -[b(t) + \sigma\gamma(t)]M(t)$$

σ 是一个正常系数，表示不规则波动的大小； $\gamma(t)$ 是一个标准高斯白噪声（White Gaussian Noise）函数。在 $M(0)=m_0$ 的情况下，可以得到方程的解

$$M(t) = m_0 \exp\left[-\int_0^t b(t)dt - \sigma W(t)\right]$$

2.1 软件可靠性

4. 新型云环境下的软件可靠性模型

戴元顺 (YuanShun. Dai) 等人于2016年提出了一种层级的相关模型，用于评估云服务的可靠性、性能和能源消耗等质量属性。

层级的相关模型

资源层

应用层

管理层

该建模方法不仅考虑了虚拟机的失效，还考虑了由于物理服务器发生故障而导致云服务失效的情形。

云服务的可靠性可以定义为其上部署的所有虚拟机均不发生故障，则可表示为

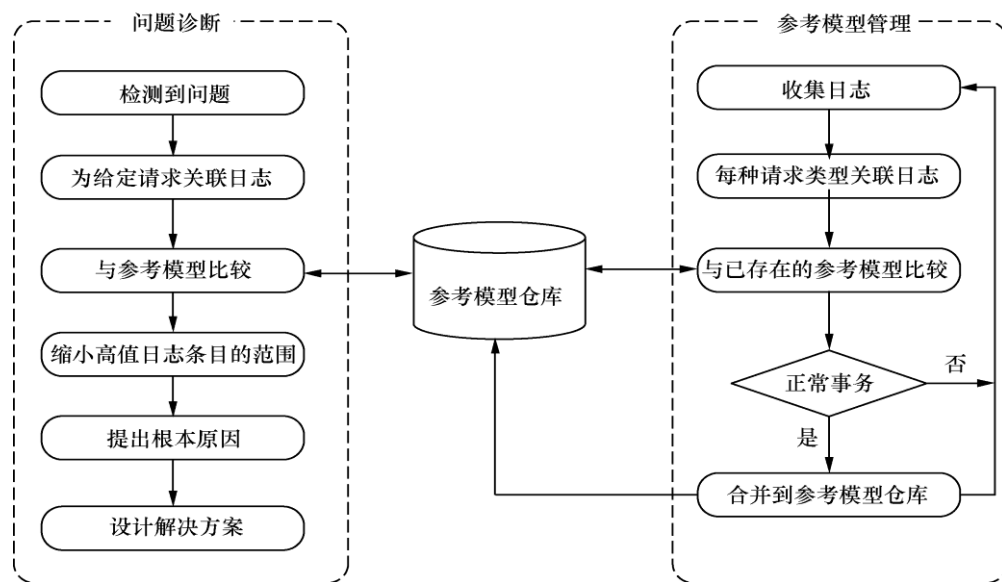
$$R = 1 - P_{\alpha}(0)$$

2.2 软件故障分析和诊断

为了快速地从在复杂的云软件运行中收集的数据诊断出软件故障，IBM相关研究组在2016年提出了一项名为日志分析（Log Analytics, LOGAN）的新方法。

LOGAN方法的主要贡献

- 通过日志，抽取正常服务情况下相关系统组件的行为，建立参考模型；
- 通过对比参考模型和实际执行的日志记录，以确定故障发生来源；



使用参考模型进行故障诊断的过程

目 录

CONTENTS

01

云测试概念

02

云可靠性度量

03

云平台的安全测试及安全
度量

04

小结

3.1 安全性测试方法

几种典型云平台存在的安全漏洞

Dropbox	iCloud	CloudMe	Flickr	Salesforce
XSS堆栈(CWE-79)	SSL 2.0(CDE-326)	CRLF注入(CDE-113)	XSS攻击(CWE-79)	CRLF 注入(CDE-113)
源代码泄露(CDE-540)			SQL盲注(CDE-89)	基于文档对象模型的XSS攻击(CWE-79)
HTML表单设有 CSRF 保护(CWE-352)		HTML表单设有 CSRF保护(CWE-352)	HTML表单设有 CSRF保护(CWE-352)	HTML表单设有 CSRF保护(CWE-352)
以明文形式发送凭证(CWE-319)		以明文形式发送凭证(CWE-319)		

3.1 安全性测试方法

1. 功能验证

功能验证是采用软件测试中的黑盒测试方法，对涉及安全的软件功能，如用户管理模块、权限管理模块等进行测试

2. 安全漏洞扫描

安全漏洞扫描通常都是借助于特定的漏洞扫描器完成的。漏洞扫描器是一种自动检测远程或本地主机安全性弱点的程序。

3. 模拟攻击实验

模拟攻击测试是一组特殊的黑盒测试用例，以模拟攻击验证软件或信息系统的安全防护能力。

模拟攻击实验类型

(1) 冒充：一个实体假装成一个不同的实体。冒充常与某些别的主动攻击形式一起使用，特别是消息的重演与篡改。

① 口令猜测：

② 缓冲区溢出：

3.1 安全性测试方法

(2) **重演**：当一个消息或部分消息为了产生非授权效果而被重复时，出现重演。

(3) **消息篡改**：数据所传送的内容被改变而未被发觉，并导致非授权后果。

① DNS高速缓存污染 ② 伪造电子邮件

(4) **服务拒绝**：当一个实体不能执行它的正常功能或它的动作妨碍了别的实体执行其正常功能的时候，便发生服务拒绝。

- | | |
|---------------------------|-------------|
| ① 死亡之Ping (ping of death) | ⑥ Smurf攻击 |
| ② 泪滴 (Tear Drop) | ⑦ Fraggle攻击 |
| ③ UDP洪水 (UDP Flood) | ⑧ 电子邮件炸弹 |
| ④ SYN洪水 (SYN Flood) | ⑨ 畸形消息攻击 |
| ⑤ Land攻击 | |

(5) **内部攻击**：当系统的合法用户以非故意或非授权方式进行动作时就称为内部攻击。大多数已知的计算机犯罪都与使系统安全遭受损害的内部攻击有密切关系。

3.1 安全性测试方法

(6) 外部攻击：外部攻击可以使用的方法有搭线（主动的与被动的）、截取辐射、冒充为系统的授权用户、冒充为系统的组成部分、为鉴别或访问控制机制设置旁路等。

(7) 陷阱门：当系统的实体受到改变，致使一个攻击者能对命令或对预定的事件、事件序列产生非授权的影响时，其结果就称为陷阱门。

(8) 特洛伊木马：对系统而言的特洛伊木马是指它不但具有自己的授权功能，而且有非授权功能。一个向非授权信道复制消息的中继就是一个特洛伊木马。典型的特洛伊木马有 NetBus、BackOrifice和BO2k等。

(9) 侦听技术：侦听技术实际上是指在数据通信或数据交互的过程中，对数据进行截取分析的过程。目前最为流行的侦听技术是网络数据包的捕获技术，通常我们称为Capture。

3.2 安全性测试方法举例

安全测试两个主要的挑战

- 生成值（也叫做测试有效载荷），其目的是实施测试；
- 评估这些测试有效载荷可以暴露一个真实的漏洞

本章节主要阐述一种应对SQL注入攻击的安全测试方法

1. SQL注入攻击

SQLi（SQL注入）漏洞是基于Web的软件系统的主要安全威胁之一。当攻击者提供包含SQL代码片段的输入值时，它们最终会注入到在数据库上执行的SQL查询中。

3.2 安全性测试方法举例

下面介绍一个SQL注入攻击的案例

在第1条SQL语句中，当攻击者注入重言式 $1=1$ 将select语句变为图所示的样式后，执行查询则可获得hotelList表中的所有记录，

```
$sql="select * from hotelList where country =' " ;
```

```
$sql = $sql . $country;
```

```
$sql = $sql . "' " ;
```

```
$result = mysql_query ($sql) or die (mysql_error () );
```

攻击后



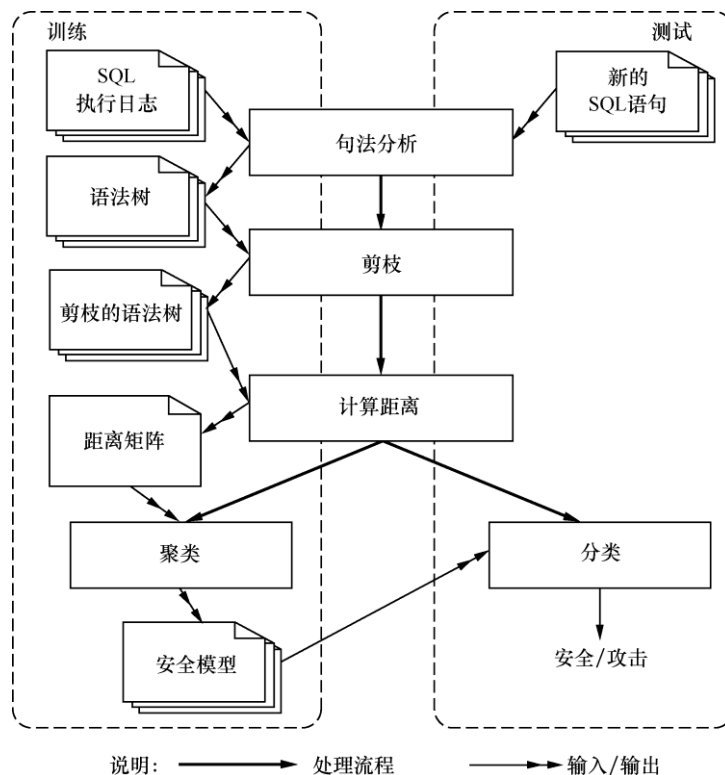
```
Select * from hotelList where country = '' or 1 = 1 --'
```

2. SOFIA攻击

SOFIA (Sececurity Oracle for SQLi At-tacks) 是一个安全Oracle (Security Oracle)，专门针对SQLi攻击。它是由一类机器分类方法构建而成的。SOFIA方法分为两类：训练和测试。

3.2 安全性测试方法举例

构建和应用SoFIA进行SQL注入安全测试的整个流程如图所示



主要根据“树编辑距离”对剪枝过的树进行聚类，“树编辑距离”大的分为一类，树编辑距离小的分为一类。当有新的SQL语句被归入现有集群中时，则认为它是良性语句，否则新的SQL语句被认为有潜在的攻击。

3.2 安全性测试方法举例

对检测到的SQL语句执行SOFIA过程

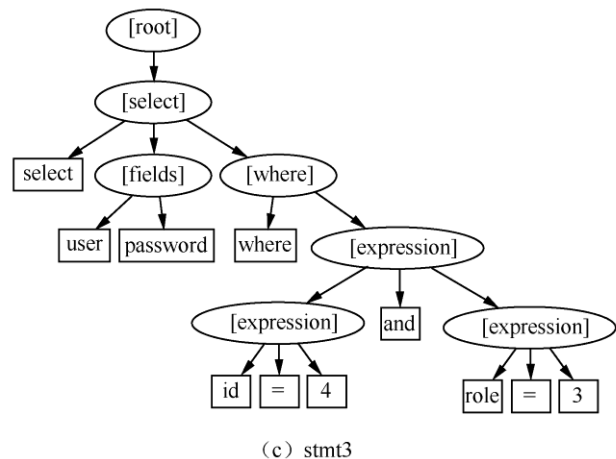
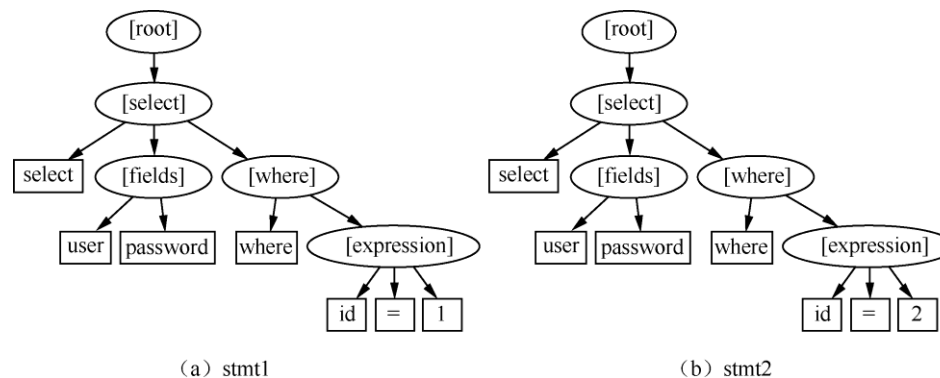
步骤1：解析。对上述3条SQL语句进行解析

Stmt1: select user, password from users

Where id = 1;

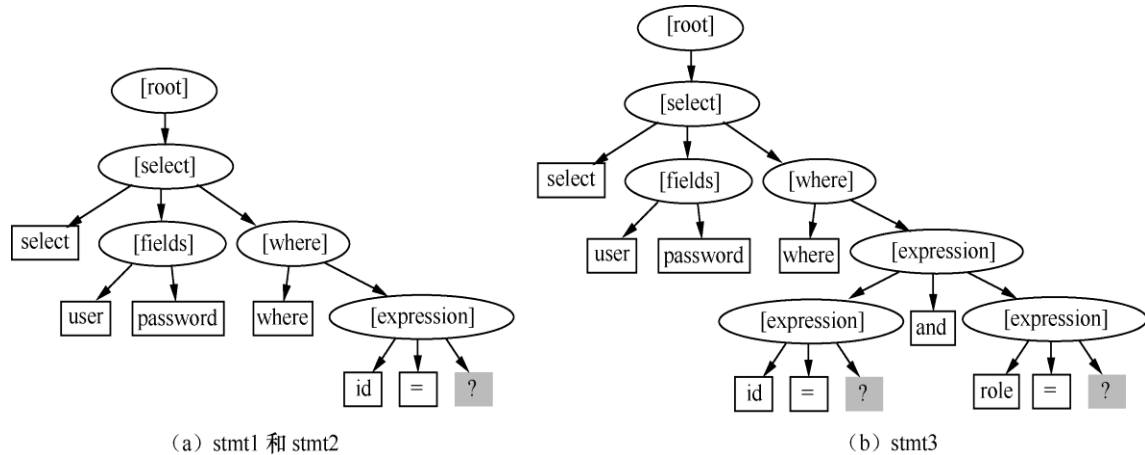
Stmt2: select user, password from users

Where id = 2;



3.2 安全性测试方法举例

步骤2：剪裁。替换所有常量数值和字符串



对解析结果进行剪裁

步骤3：计算树编辑距离。利用approxlib工具，该工具实现了计算树编辑距离的算法，但利用树编辑距离进行分类，往往是不够的。

步骤4：聚类。利用K-中心算法进行分类，分类结果如表所示。

Cluster	Medoid	Elements
1	stmt1	stmt1, stmt2
2	stmt3	stmt3

目 录

CONTENTS

01

云测试概念

02

云可靠性度量

03

云平台的安全测试及安全
度量

04

小结

小结

这个知识点主要学习内容：

- 云测试概念
- 云测试的基本理论
- 云可靠性度量
- 云平台的安全测试及安全度量